Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación

Construcción de Compiladores

Proyecto 2

Generación de Código Intermedio para Compiscript

Documentación de arquitectura y ejecución

Compiler Toolkit

Estudiante
Osman Emanuel de León García
Carné 23428

Guatemala, octubre de 2026

Índice

1. Introducción y alcance **2**

2. Arquitectura general **2**

3. Diseño del código intermedio **2**

3.1. Instrucciones principales 3

3.2. Ejemplo de traducción 3

4. Generación de TAC y temporales **3**

4.1. Control de flujo 3

4.2. Funciones, cierres y estructuras 4

5. Tabla de símbolos y registros de activación **4**

6. Interfaz gráfica e inspección de resultados **4**

7. Instalación y ejecución **5**

8. Batería de pruebas y resultados **5**

9. Supuestos y límites conocidos **6**

10. Conclusiones **6**

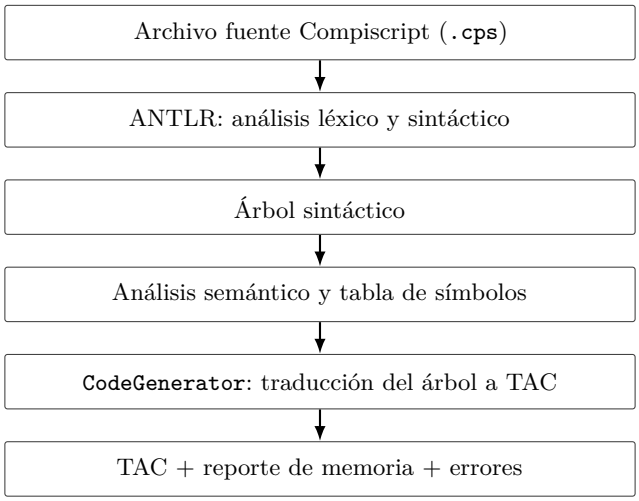
1. Introducción y alcance

Compiler Toolkit integra las herramientas YALex y YAPar desarrolladas en las etapas anteriores del curso, junto con el compilador de Compiscript. En esta entrega, el trabajo se concentra en la **generación de código intermedio**: a partir del árbol sintáctico de ANTLR y de la información preparada por el análisis semántico, se produce una representación de tres direcciones (TAC).

El alcance incluye la definición del lenguaje intermedio, traducción de expresiones y estructuras de control, funciones, clases, listas, administración de temporales y reporte de la memoria asociada a símbolos y registros de activación. La interfaz gráfica integra estas salidas para permitir la compilación y la inspección de resultados desde archivos `.cps`.

2. Arquitectura general

El flujo de Compiscript separa las fases para que cada una trabaje con el resultado de la anterior. La generación TAC se ejecuta solo si los análisis léxico, sintáctico y semántico terminan sin errores.



Los módulos principales de esta etapa son:

Archivo	Responsabilidad
compiscript/tac.py	Define operandos e instrucciones TAC y su formato de salida.
compiscript/codegen.py	Recorre el árbol de ANTLR y genera las instrucciones de cada construcción.
compiscript/temps.py	Asigna y recicla variables temporales por función.
compiscript/symbol_table.py	Mantiene los símbolos, ámbitos y datos de almacenamiento.
compiscript/activation_record.py	Calcula y presenta datos globales, disposición de objetos y registros de activación.
compiscript/semantic.py	Valida el programa y anota tipos, ámbitos y símbolos para el generador.
gui/app.py	Integra editor, compilación, resultados TAC y reporte de memoria.
tests/codegen/	Contiene pruebas de generación y una máquina virtual mínima para validar resultados.

3. Diseño del código intermedio

El TAC utiliza instrucciones simples de asignación, operaciones, saltos, llamadas y acceso a estructuras. Cada función se representa como un bloque independiente; el programa principal se denomina `main`. Las etiquetas de salto se identifican como `L1`, `L2`, etc.

3.1. Instrucciones principales

Forma	Significado
x = y; x = y op z	Copia y operaciones binarias (aritmética o comparación).
x = -y; x = !y	Negación aritmética y lógica.
L:; goto L	Define etiqueta y realiza un salto incondicional.
if x goto L; ifFalse x goto L	Saltos condicionales.
if x op y goto L	Salto según una comparación relacional.
param x; call f, n	Pasa argumentos y llama a una función con `n` parámetros.
x = call f, n; return x	Recoge el retorno de una llamada y retorna un valor.
x = a[i]; a[i] = x	Lectura y escritura de listas por índice.
x = o.campo; o.campo = x	Lectura y escritura de atributos de objetos.
x = new C, n; x = newarray n	Crea objetos y listas.
enter n; halt	Reserva el registro de activación y termina el programa principal.

La concatenación de cadenas utiliza la operación `concat`. La llamada a `print` se representa con el mecanismo general de parámetros y llamadas. Cada valor ocupa una palabra de cuatro bytes según los supuestos de esta implementación.

3.2. Ejemplo de traducción

Para el programa:

```
let a: integer = 7;
let b: integer = 3;
print(a + b * 2);
```

Una salida TAC representativa es:

```
func main: ; nivel 0, registro de 20 bytes
    enter 20
    a = 7
    b = 3
    t0 = b * 2
    t0 = a + t0
    param t0
    call print, 1
    halt
```

La multiplicación se calcula antes que la suma para respetar la precedencia. El temporal puede reutilizarse después de consumir su valor anterior, por lo que el generador no necesita conservar un nombre temporal distinto para cada operación.

4. Generación de TAC y temporales

`CodeGenerator` implementa el patrón Visitor para traducir los nodos del árbol. Mantiene un contexto por función, con su código, pool de temporales, nivel léxico y pila de destinos para los ciclos. Para el control de flujo genera etiquetas y saltos; para expresiones lógicas emplea saltos condicionales que permiten preservar el cortocircuito de `&&`, `||` y `!`.

El pool de `compiscript/temps.py` asigna el temporal libre de menor índice o crea uno nuevo si no existe. Una vez consumido un valor temporal, se libera para poder reutilizarlo. En expresiones cuyo resultado se asigna inmediatamente, el generador puede escribir directamente en la variable destino y evitar una copia adicional.

4.1. Control de flujo

Las construcciones `if/else`, `while`, `do-while`, `for` y `switch` se traducen con etiquetas y saltos. Una pila de contextos de ciclo permite que `break` salte al final del ciclo y `continue` se dirija al punto apropiado de la siguiente iteración.

4.2. Funciones, cierres y estructuras

Las funciones se generan como bloques separados con instrucciones `param`, `call` y `return`; la recursividad utiliza el mismo convenio de llamadas. Para funciones anidadas, un enlace de acceso permite leer o modificar variables del ámbito que las contiene. El TAC expresa ese acceso como `x@k`, donde `k` indica los enlaces léxicos que deben recorrerse.

Las clases y los objetos se representan mediante instrucciones de creación, acceso a atributos e invocación de métodos. Los offsets de los campos mantienen la disposición heredada. Las listas se crean con `newarray`; su tamaño se consulta mediante `len` y sus elementos se leen o escriben mediante indexación.

Al terminar una función, el generador aplica limpiezas de código: elimina instrucciones inalcanzables, simplifica ciertos saltos y elimina etiquetas sin uso. Se repiten hasta estabilizar el bloque.

5. Tabla de símbolos y registros de activación

La tabla de símbolos incluye la información necesaria para localizar datos durante la generación: tipo, categoría de almacenamiento, offset, tamaño, etiqueta, profundidad léxica y tamaño del marco de función.

`Campo`	Uso
storage	Distingue variables globales/locales, parámetros, campos, funciones, métodos, clases y elementos predefinidos.
offset	Posición en bytes dentro de datos globales, registro de activación u objeto.
size	Tamaño reservado; los valores y referencias usan una palabra de cuatro bytes.
label	Nombre interno para identificar funciones y métodos en el TAC.
depth	Profundidad del ámbito donde se declara un símbolo; apoya el acceso léxico.
frame_size	Tamaño calculado del registro de activación de una función, antes de sumar los temporales usados.

El registro de activación utiliza estos campos iniciales:

`Offset`	Contenido
fp+0	Valor de retorno
fp+4	Dirección de retorno
fp+8	Enlace de control: marco del llamador
fp+12	Enlace de acceso: entorno léxico de la función
fp+16 en adelante	Parámetros, variables locales y temporales

En métodos, `this` se trata como el primer parámetro. Las variables de bloques anidados ocupan espacio del marco que las contiene; bloques hermanos pueden reutilizar posiciones y el tamaño final refleja el máximo uso. El reporte se muestra en la pestaña **Memoria CPS** y puede inspeccionarse también desde la línea de comandos.

6. Interfaz gráfica e inspección de resultados

La interfaz desarrollada en `gui/app.py` permite editar Compiscript y ejecutar la compilación desde una misma ventana. En las capturas de ejecución proporcionadas para la revisión se observan el editor, los botones de compilación y prueba, la pestaña **TAC CPS** con instrucciones generadas y la pestaña **Memoria CPS** con offsets globales y el registro de activación de `main`.

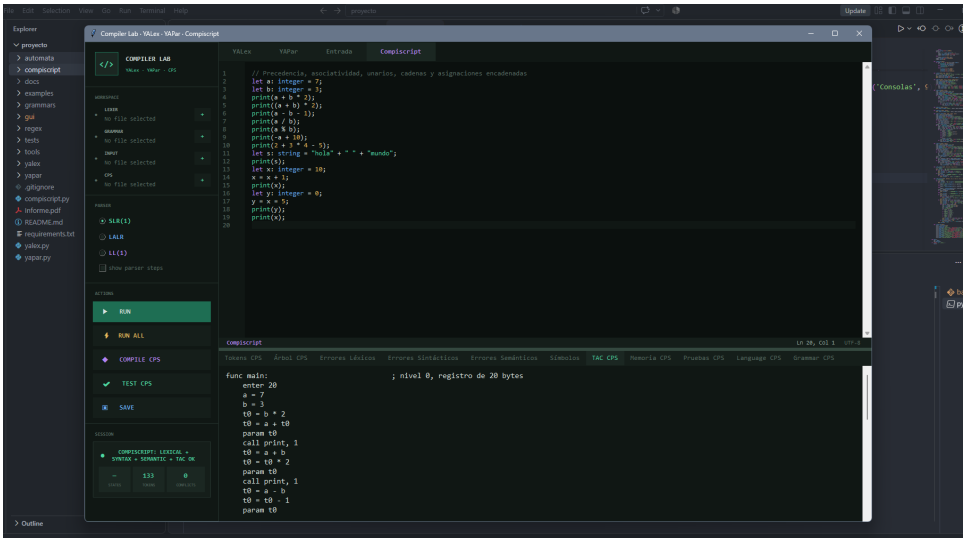


Figura 1: Vista del IDE integrado con las salidas TAC y de memoria.

7. Instalación y ejecución

El entorno usado para las pruebas reportadas fue Python 3.11.0 sobre Windows. Desde la raíz del repositorio:

```
git clone https://github.com/Emadlgg/compiler-toolkit.git
cd compiler-toolkit
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

Para abrir la interfaz:

```
python gui/app.py
```

Para compilar desde la línea de comandos y ver las salidas relevantes:

```
python compiscript.py programa.cps --tac
python compiscript.py programa.cps --symbols
python compiscript.py programa.cps --frames
python compiscript.py programa.cps --all
```

8. Batería de pruebas y resultados

La batería `tests/codegen/run_tests.py` compara el TAC generado con salidas esperadas y ejecuta los programas válidos en una máquina virtual mínima de pruebas para contrastar su salida. También incluye casos inválidos que verifican diagnósticos y una prueba unitaria del pool de temporales.

El resultado compartido de la ejecución local del 16 de octubre de 2026 fue:

Grupo	**Resultado**
Prueba unitaria de temporales	1/1
Programas válidos	13/13
Programas con errores esperados	10/10
Total	**24/24 correctas**

Los casos válidos abarcan expresiones, operaciones lógicas, control de flujo, `switch`, funciones, cierres léxicos, clases, listas, ternarios, temporales, variantes de `for`, constantes y herencia. Los casos inválidos cubren errores de tipos, declaraciones, llamadas, flujo, listas, clases, código inalcanzable, construcciones no soportadas y sintaxis. Este resultado confirma la aprobación de los casos incluidos en la batería, no una verificación exhaustiva de todo programa posible.

9. Supuestos y límites conocidos

La implementación adopta los siguientes supuestos: los valores y referencias ocupan cuatro bytes; la evaluación de expresiones y argumentos sigue el orden especificado en `docs/CI.md`; los campos de objeto se disponen con offsets y la herencia conserva los offsets de los atributos base; y el generador solo se ejecuta cuando las fases previas no reportan errores.

Hay construcciones que el generador reporta explícitamente como no soportadas: `try/catch`, usar un nombre de función o método como valor y llamar al resultado de otra llamada (por ejemplo, `f()()`). Las funciones anidadas acceden a su entorno mediante enlaces léxicos, pero no son valores de primera clase; por ello, el alcance de closures está limitado a funciones llamadas mientras el entorno que las define continúa activo. El despacho de métodos se resuelve según el tipo estático del receptor, y la gramática no contempla una llamada explícita a `super` desde el constructor.

10. Conclusiones

La implementación integra generación TAC, administración de temporales y reporte de almacenamiento sobre el front-end existente de Compiscript. La definición de instrucciones está separada de la lógica de traducción, y la tabla de símbolos aporta la información requerida para ubicaciones, objetos y registros de activación. La ejecución reportada de 24 pruebas correctas y la inspección de las pestañas TAC CPS y Memoria CPS aportan evidencia de funcionamiento para los casos incluidos.

Repositorio: `https://github.com/Emadlgg/compiler-toolkit`

Referencias

[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2.ł ed., Pearson, 2006.

[2] A. W. Appel, *Modern Compiler Implementation in Java*, 2.ł ed., Cambridge University Press, 2002.

[3] K. D. Cooper y L. Torczon, *Engineering a Compiler*, 2.ł ed., Morgan Kaufmann, 2011.

[4] T. Parr, *The Definitive ANTLR 4 Reference*, Pragmatic Bookshelf, 2013.